Deep Learning Approach to Malicious Network Traffic Classification

Kaan Aydogan, Jacob Clements, Cameron Andrews, Amadeo De La Vega
Department of Computer Science
University of Maryland
College Park, MD 20742
{kaan, clem012, andrewsc, adelaveg}@umd.edu

1 Introduction

From spam to brute force to DDoS and other attacks, it is estimated by the Bad Bot Report that approximately 27% of internet traffic in 2022 was made up of malicious bot traffic [2]. It is imperative that public facing servers take measures to identify and deter these attacks to protect sensitive data. Services such as online banking are essential to the economy and rely on cybersecurity to function. But not only that, as technology advances and our reliance on digital systems grows, the need to identify and mitigate malicious network activity becomes increasingly crucial. The impact of such threats can be devastating, ranging from unauthorized access to sensitive information to widespread disruption of critical infrastructure. Therefore, effective detection mechanisms and proactive defense strategies are vital to safeguarding digital assets and maintaining a secure cyberspace.

In this project, we use deep learning techniques to detect and deter malicious network traffic. Our group shared an interest in a project in the cybersecurity space. We wanted to pursue this project specifically because we believed it provided an opportunity where it was feasible to obtain practically measurable results, having a final product that is a demonstration of a real-world cybersecurity tool.

1.1 Traffic Classification Approaches

Conventional packet filtering by modern firewalls is based on the limited unencrypted information in packet headers. Our goal was to use deep learning to infer characteristics past this basic information, to classify the malice of a packet on a more comprehensive basis. Practicality of our model was one our emphasized goals, and we wanted to build something that could be feasibly integrated to classify packets in real time. We looked into two different approaches to do so, eventually focusing our efforts on one.

1.1.1 Flow-Based Classification

Taking a flow-based approach would mean analyzing network traffic patterns over time, and identifying certain patterns that indicate security threats. Many network traffic datasets provide flow based features in addition to raw data, and there are various open source tools to generate certain such features. We found through our work that a problem with flow-based analysis was the lack of standardization in the features provided across different datasets, and difficulty to replicate these features. This is discussed in more detail in Section 2. The lack of standardization in data, and the dependence of each packet on previous packets made it harder for us to practically apply a model we made using this approach.

1.1.2 Payload-Based Classification

The Payload-Byte paper acknowledges the lack of standardization in network traffic datasets, and the resulting reproducability issues [1]. They instead outline an approach to extract information from standard raw packet payload data. A majority of network traffic datasets regardless of their feature

extraction also provide raw network data, making it feasible to train a model based on the payload data of popular datasets. Given our goals and the information in this paper, we decided to focus our efforts on payload-based packet classification.

2 Data

We chose to use data from the UNSW-NB15 network traffic dataset to develop our model [3]. Due to the surplus of data avaialable in this set, for consistency and reproducability, we decided it was best to focus on developing our model using only this dataset. The decision to use one dataset was made before the decision to focus on a more standard payload-based classification, and we are open to expanding our sources of data in future iterations of this project.

2.1 Dataset Description

The UNSW-NB15 dataset created by the University of New South Wales in Australia contains network traffic data with both benign and malicious requests. Being released in 2015, it is relatively new compared to other popular network security datasets. The data is derived from a combination of natural and synthetic traffic and was captured in a live network environment using the tcpdump tool. The dataset has 49 total features: primarily flow-based features extracted using the Argus utility, some additional features developed using twelve custom algorithms, and the label and attack category. Raw packet capture files are also provided.

2.2 Initial Challenges

Reproducability in a practical context being one of our main goals, the lack of transparency in feature extraction in this dataset was a problem. In an attempt to replicate the features on our own network traffic, we used the Argus network utility to extract as many of the flow features as possible. While some of the features were easily extracted using Argus, many of them were not, presumably having been extracted by one of the twelve custom algorithms discussed in their paper [3]. Our options were to either accept this and fill in missing features with medians or K-Nearest-Neighbors, or represent the data in a more standard way.

3 Related Work

The two papers with the heaviest influence on our project include a paper on byte sequences in malware detection by Yu et. al, and the Payload-Byte paper. The paper on byte sequences in malware detection concluded that 1D Convolutional Neural Networks outperformed other architectures in malware classification tasks for payloads provided in byte sequences [4]. This is what inspired us to try using this, and ultimately having the final version of our model incorporate 1D convolutions. Additionally, the Payload-Byte paper was of great use to us in this project. The paper provided inspiration to use payload-based classification, gave us tools to extract such features on our own network data, and provided a version of the UNSW-NB15 dataset that was already run through this tool [1]. Payload-Byte researched the best way to extract a standard set of features from network data, so we see our work as a form of expansion on this project by implementing a learning model using the format of data described.

4 Approach

Our approach was to first represent network packets as proper feature vectors, train deep learning models on this data, and to develop a tool that sniffs packets on a live server and runs them through the model to obtain a classification.

4.1 Representing Data

Deciding to use a payload-focused representation of a packet, meant that we had to use the raw packet capture files published as part of the dataset to construct our own feature vectors. Packet captures

generated by common tools such as tcpdump typically follow the format as outlined in Figure 1 below, with a list of packets that each contain a header with information, and payload data.

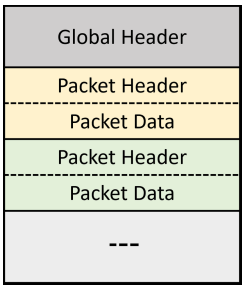

Figure 1: Common PCAP File Structure [1]

To process raw packet capture data, we used the open source Payload-Byte tool published as part of the paper previously cited [1]. This tool takes packet capture data and outputs it in the format described in the paper, with four features from the packet header, and 1500 byte-long integer features which represent the payload. The resulting feature vector is shown in Figure 2.

Figure 2: Payload-Byte Feature Vector Output [1]

We apply one-hot encoding on the protocol column of this output, representing each string protocol value as being either one of the four most common protocols in the dataset: {tcp, udp, ospf, sctp}, or {other} if it represented a miscellaneous protocol that made up less than 10% of the training data. Thus, the protocol feature was instead represented in these five dimensions, giving us a total feature vector size of 1508.

The Payload-Byte paper also includes a version of the UNSW-NB15 dataset pre-processed through their algorithm in CSV format, saving us the storage and computing necessary to run their tool on the high volume of raw network data in the dataset.

4.2 Neural Network

Throughout developing our model, we went through three different iterations of neural networks, which we will refer to as Net{1,2,3}. Net3 was the version of our model which we focused most heavily on, as it was the version that was used in our final product.

4.2.1 Net1: Preliminary Flow-Based Feedforward Network

In the first iteration of our model, we used the pre-generated features of the UNSW-NB15 dataset to guage how accurately we could learn them using a simple neural network. The final version of this network consisted of two fully connected layers, including one output layer which mapped to a binary classification. This preliminary model served as an early proof-of-concept in our network traffic modeling.

4.2.2 Net2: Payload-Based Feedforward Network

After deciding to use the features as described in Section 4.1, Net2 was another simple preliminary network to serve as a proof of the concept that we could train a deep learning model using this new form of data. We recycled the same two-layer feedforward neural network architecture for a binary classification of each packet. The model consisted of two fully connected layers: the first layer used

the ReLu nonlinearity, and the second layer used the sigmoid nonlinearity. The size of the hidden layer was 48.

4.2.3 Net3: Convolutional Network

The final version of our model utilized a neural network architecture as summarized in figure 3.

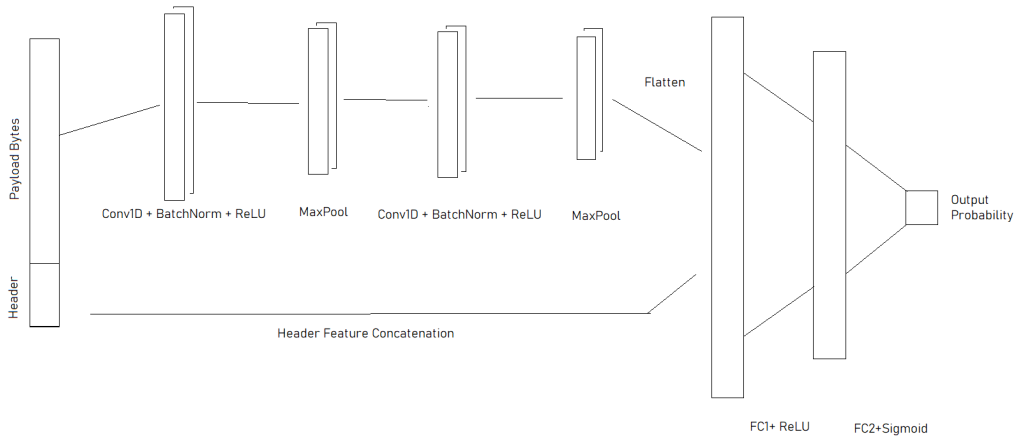

Figure 3: Net3 Diagram

The approach is to use 1D convolution filters over the packet payload to capture potential local patterns in the data. We believe it makes sense that in packet payloads divided into byte-long features, neigboring features could exhibit correlations or patterns where it would benefit to apply this convolution. Two layers of convolution are applied to the 1500 features that represent the payload, followed by Batch Normalization, ReLU, and Max Pooling. The output is flattened and concatenated with the eight header features, to create a new convolved vector representation of the data. This is then passed through two fully connected layers to output a binary classification.

4.3 Live Application

The live application portion of our project involved building an application that sniffs packets on a network, extracts our model's expected feature vector, and logs the request's details along with the model's classification. Figure 4 summarizes the flow a network request through the application to obtain a classification.

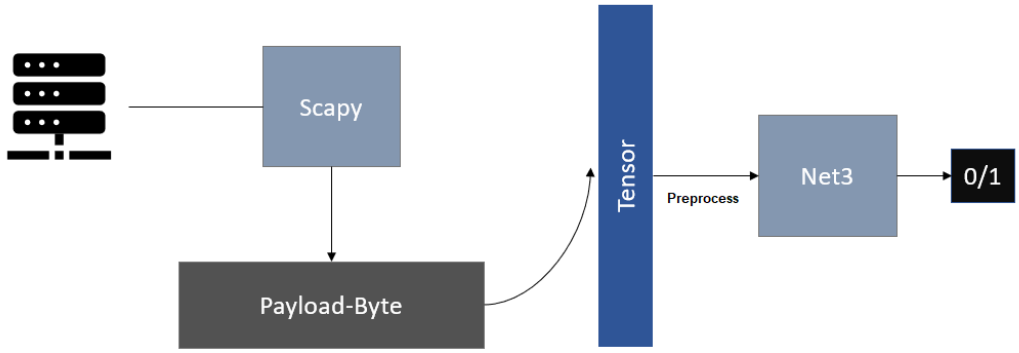

Figure 4: Live Application Diagram

The application was developed in Python, using the Scapy packet manipulation library. The library provides functionality to sniff packets, and extract their header and payload data. Accordingly, this data is passed into the functions described in the Payload-Byte application to ultimately output our model's feature vector. The model is run on this input, and the basic request characteristics along with the classification are logged. Figure 5 shows example outputs from a brief application run on a test server.

```
kaan@kaanserver:~/testing/netwatch$ sudo python3 netwatch.py
[2023-05-11 18:24:12] 162.159.130.234:443→10.0.0.166:45380, BENIGN
[2023-05-11 18:24:12] 104.18.29.120:443→10.0.0.166:59720, BENIGN
[2023-05-11 18:24:12] 162.159.135.234:443→10.0.0.166:52290, MALICIOUS
[2023-05-11 18:24:12] 23.23.72.188:27017→10.0.0.166:34600, MALICIOUS
[2023-05-11 18:24:12] 162.159.133.234:443→10.0.0.166:46906, BENIGN
[2023-05-11 18:24:12] 23.23.72.188:27017→10.0.0.166:34646, MALICIOUS
[2023-05-11 18:24:12] 162.159.130.234:443→10.0.0.166:46992, BENIGN
[2023-05-11 18:24:12] 162.159.137.232:443→10.0.0.166:43810, BENIGN
[2023-05-11 18:24:12] 162.159.137.232:443→10.0.0.166:43810, BENIGN
[2023-05-11 18:24:12] 162.159.133.234:443→10.0.0.166:43964, BENIGN
```

Figure 5: Live Application Demo Run

As seen in the capture, there were mostly HTTPS requests that were labeled benign, one malicious HTTPS request, and two other miscellaneous requests labeled as malicious.

5 Experiments and Results

5.1 Performance on Dataset

The final iteration of our model (Net3) and hyperparameters achieved 99.9% accuracy in both training and validation. We experimented with using batch normalization and dropout, finding that an architecture including batch normalization without dropout worked the best. Most of our hyperparameter tuning included increasing the batch size and decreasing the learning rate, to converge nicely to high accuracy as seen in Figure 7. Our final learning rate was 0.00001 with a batch size of 350.

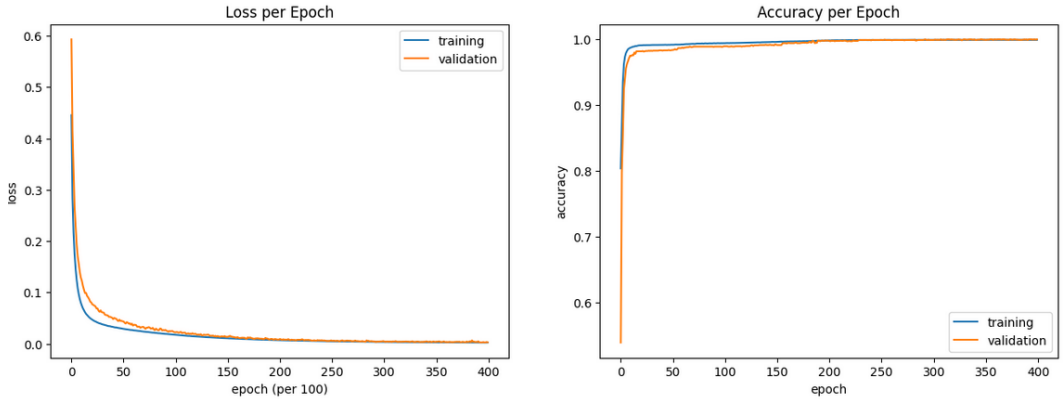

Figure 6: Loss and Accuracy plots for Net3

5.2 Performance on New Data

Our model achieved very high accuracy on our validation subset, which means it learned data from the one source very well, however we needed to guage how well it can generalize to data from completely different origins.Using roughly the same method as described in the application, we ran our model on real-world packet captures to get an idea of the practical performance of our model. Upon first testing on new data, we found that it classified an unusually high amount of packets as malicious regardless of the reality. We realized that our training set consisted of more malicious than benign examples, feeding the model an unrealistic distribution. We decided to fabricate the training and validation subsets differently, such that we validated on equal proportions, however trained the model on a skewed split with more benign examples. This way, it learns from a more representative set, but is validated on both equally. This was slightly tuned to a final 20/80 split between benign/malicious examples respectively.

We also ran inference on four packet captures which were all expected to consist of either mostly malicious or mostly benign packets. The four runs as summarized:

- Capture 1 was a DDOS attack packet capture that we downloaded from a StopDDOS GitHub repository: https://github.com/StopDDoS/packet-captures. The model classified 48% of the packets as malicious.
- Capture 2 was a DDOS attack that we fabricated on a test server, using a simple open source DoS tool found at https://github.com/gkbrk/slowloris, and capturing ten minutes worth of network data during attack. The model classified 33% of the packets as malicious.
- Capture 3 was a capture of regular activity on one of our test servers. Notably, this test server serves a public application with usage. The model classified 17% of the packets as malicious.
- Capture 4 was a capture of regular activity, limited to HTTPS traffic, on a test server. The model classified 12% of the packets as malicious.

The trend is that the captures we expect to have more malicious packets, are classified as more heavily malicious. We are currently not able to conclude the statistical significance of these results and would need to run more comprehensive tests using similar techniques. However, we do take the positive trend as a sign that our model does not solely overfit to our dataset, displaying some practical power.

6 Conclusion and discussion

We used deep learning techniques in this project to address the critical problem of classifying malicious network traffic. We became aware of the growing importance of detecting and preventing malicious activity in order to protect private information, maintain cybersecurity, and ensure the availability of online services. Our goal was to develop a practical and efficient method for detecting

and classifying malicious network traffic in real-time. After investigating various traffic classification approaches, including flow-based and payload-based methods, we decided to concentrate our efforts on payload-based classification. We used the approach described in the Payload-Byte paper to extract information from raw packet payload data, which provided a more standardized representation and better reproducibility than flow-based analysis. Using the UNSW-NB15 dataset, we created Net3, a neural network model that used 1D convolutional layers to detect local patterns in the packet payload. Using this method, we were able to effectively classify packets as benign or malicious. Using the dataset, we trained and evaluated the model, achieving high accuracy and demonstrating the feasibility of our approach. We also created a live application that used our trained model to classify network traffic in real-time. The application sniffed network packets, extracted required features, and provided classifications and request details. This live testing demonstrated the model's applicability and effectiveness in a real-world scenario.

Struggles in our project initally came when we were developing our model on the flow-based features of the UNSW-NB15 dataset. Although our model performance was fine, being able to test on real-world data was a big emphasis of this project, and we came short of being able to extract the same features. After reading through more papers and approaches, coming across the Payload-Byte tool greatly helped us develop a final product that we had envisioned. Ensuring and judging practical accuracy of our model still remains one of the difficult parts of this project. Although our group is interested in cybersecurity and proficient in some networking concepts, having an advanced knowledge of packet flows and attacks could definitely help us identify places in which we could improve to generalize past our training data. The results on outside data provided us promise, however we know that we need to look into techniques to help the model generalize before our tool could be reliably used in a production environment.

7 Future Work

For future expansions of this project, we could consider modifying the model to be able to classify the attack type if a packet is deemed malicious. We would also look for other datasets to combine with our current one, which could help in generalization to improve the real-world performance of our model. Building upon this, we would employ more comprehensive techniques to guage the performance on new data, running on larger samples of real-world packet captures, and using statistical techniques to determine the significance of the results.Finally, we can improve the application portion of our project by also incorporating functionality to block malicious requests.

References

[1] Yasir Ali Farrukh, Irfan Khan, Syed Wali, David Bierbrauer, John A. Pavlik, and Nathaniel D. Bastian. Payload-byte: A tool for extracting and labeling packet capture files of modern network intrusion detection datasets. In *2022 IEEE/ACM International Conference on Big Data Computing, Applications and Technologies (BDCAT)*, pages 58–67, 2022.

[2] Imperva. 2022 imperva bad bot report. `https://www.imperva.com/resources/resource-library/reports/bad-bot-report/`, 2022.

[3] Nour Moustafa and Jill Slay. Unsw-nb15: a comprehensive data set for network intrusion detection systems (unsw-nb15 network data set). In *2015 Military Communications and Information Systems Conference (MilCIS)*, pages 1–6, 2015.

[4] Tong YU, Xin LI, Ying LING, Dongmei BIN, and Chunyan YANG. Deep learning based malware traffic classification for power internet of things network security. In *2022 7th International Conference on Big Data and Computing*. ACM, May 2022.